

HOW TO TRAIN YOUR INSTRUCTION-FOLLOWING TEXT ENCODER WITHOUT LABELING

JIATONG YU

ADVISOR: PROFESSOR SANJEEV ARORA

SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE DEGREE OF
BACHELOR OF ARTS
DEPARTMENT OF COMPUTER SCIENCE
PRINCETON UNIVERSITY

APRIL 2024

I hereby declare that I am the sole author of this thesis.

I authorize Princeton University to lend this thesis to other institutions or individuals for the purpose of scholarly research.

Jiatong Yu

I further authorize Princeton University to reproduce this thesis by photocopying or by other means, in total or in part, at the request of other institutions or individuals for the purpose of scholarly research.

Jiatong Yu

Abstract

Text embedding models encode the semantic content of natural language inputs into fixed-length vectors. Contrastive learning has been the go-to training strategy to ensure semantically similar vectors are closely positioned in the embedding space. While successful, this training recipe requires large amounts of labeled training data in order to cover diverse domains. In addition, current text embedding models cannot adhere to user instructions when encoding inputs. In this thesis, we provide the first attempt to build text embedding models that can (1) adhere to user instructions, and (2) generalize without domain-specific labeled data. We leverage the recent success of generative large language models (LLMs), which exhibit strong domain generalization and rich latent knowledge. Instruction-tuning methods have further matured in the past two years, allowing language models to adhere to user instructions and to perform unseen tasks. Transferring these properties to text encoders can enrich their contextualized representations, boost domain robustness, and increase retrieval reliability in performance-critical settings. In this work, we rely on the state-space model (SSM) parametrization to achieve the aforementioned goals. In control theory, SSMs are defined through ordinary differential equations with respect to the state vector, capturing the dynamics of an information system over time. State vectors are a fixed-length compression of the system’s past trajectory. We show that, empirically, state vectors in learned, discretized SSMs still preserve this information-rich property in language modeling tasks, and can be applied off-the-shelf to perform text embedding tasks. The instruction-following ability of pretrained generative LMs allows the state vectors to be sensitive to user intentions and can successfully compress different information, given different prompts.

Acknowledgements

I would like to thank Professor Sanjeev Arora for advising my thesis. I am also grateful to Professors Danqi Chen and Kai Li for advising me in related lines of work. I thank Michael Poli for providing insightful feedback on this project. Last, I would like to thank my friends and family, and everyone who has supported me at Princeton University.

Contents

Abstract	iii
Acknowledgements	iv
List of Tables	vii
List of Figures	viii
1 Introduction	1
1.1 Motivation	2
1.2 Outline	4
1.3 Contributions	4
2 Background and Related Work	6
2.1 State Space Models	6
2.2 SSMs for Sequence Modeling	8
2.3 Retrieval-Augmented Generation	11
3 Analyzing State Vectors as Embeddings	16
3.1 Baseline Embedding Performance	17
3.1.1 Textual Encoder from Pretrained SSMs	17
3.1.2 Evaluation Benchmark	18
3.1.3 Results and Discussion	20
3.2 Locating Factual Knowledge in State Vectors	22
3.2.1 Activation Patching	22

3.2.2	Experiment Setup	25
3.2.3	Results and Discussion	26
3.3	Semantic and Lexical Perturbation	27
3.3.1	Sensitivity to Sentence Completeness	28
3.3.2	Sensitivity to Semantic and Structural Similarity	29
3.3.3	Results and Discussion	30
4	Aligning Embedding Space through Instructions	32
4.1	Introduction	33
4.1.1	Background	33
4.1.2	Promblem Definition	34
4.2	Instruction Prompting Setup	35
4.3	Results and Discussion	37
5	Conclusion	40
5.1	Overview	40
5.2	Limitations and Future Work	41

List of Tables

3.1	Baseline embedding performance on the HagridEval dataset.	21
3.2	Semantic perturbation results.	30
4.1	Examples of zero-shot instruction prompting.	34
4.2	Semantic perturbation results with prompting.	38

List of Figures

3.1	Illustration of HAGRID data instance (left). HagridEval creation process (right).	18
3.2	HagridEval embedding performance by layer.	22
3.3	StripedHyena-7B. Orange lines represent the embedding score of short convolution filters, while blue lines represent SSM state vectors.	23
3.4	Visualization of state collapse behavior in Mamba across sequence length. Channels are randomly sampled.	24
3.5	Simplified illustration of activation patching.	25
3.6	Illustration of state-space parameterized layers in Mamba.	26
3.7	Activation patching results in Mamba-2.8b	27
3.8	Illustration of sentence completeness and its impact to expected next token, measured by mamba-2.8b	28
3.9	Structural and semantic similarity weighing.	31
4.1	Illustration of Instruction-Controlled Embedding.	35
4.2	t-NSE visualizations of instruction-controlled representations, topic	36
4.3	t-NSE visualizations of instruction-controlled representations, sentence type	37
4.4	Instruction prompting effect on HagridEval, by layer.	39

Chapter 1

Introduction

Generating text, reasoning about text, and retrieving information from text - these three tasks are inextricably linked, and lie at the heart of modern AI research and the recent rise of large language models (LLMs). But, despite their fundamental similarities, these steps are performed by separately trained modules, with separate architectures, weights, and learning objectives. If an AI system is powerful enough to perform one of these tasks, why can't its hidden activations be used for the others?

We have witnessed rapid growth in the complex reasoning and generalization abilities of large language models (LLMs), thanks to their encoded latent knowledge obtained through massive pretraining. These pretrained models can learn new tasks[3], adhere to human instructions[29], and alter their reasoning behavior[40] through in-context natural language descriptions. These abilities allow LLMs to serve as general-purpose chat assistants. However, standalone LLMs fall short in more targeted and information-dense domains such as medical diagnosis and legal consultation, as these domains require constant factual updates. We need systems that can traverse, process, and root their reasoning in large volumes of external information – what is commonly referred to as the retrieval-augmented generation (RAG) pipeline. The ability to *retrieve relevant information* and *efficiently process extra-long sequences* of information are two core abilities in this pipeline.

But current solutions to both of these requirements remain unsatisfying.

1.1 Motivation

Contextualized Text Embeddings The accuracy of retrieving relevant information is determined by how information is represented in a database. In particular, natural language documents are expected to be summarized by fixed-length embedding vectors. A predetermined similarity function $g(\cdot)$ is equipped along with the text encoder model to determine the relevancy between two vectorized representations of natural language inputs. Cosine similarity is the most common choice of $g(\cdot)$. In general, text encoders are trained to position semantically similar texts near each other in the embedding space, therefore cosine similarity suffices as a relevancy measurement. To ensure this spatial property, text encoders are trained through *supervised-learning*, predominantly using the contrastive learning objectives. With supervised learning comes the need of labeled data. Unlike generative LM pretraining which only involves mining and filtering natural language sequences, supervised learning requires both massive datasets to cover diverse domains and high-quality labeling of these datasets. This data-level challenge makes text encoders evolve much slower than their generative LMs counterparts. One may ask, what about distilling text encoders from pretrained generative LMs? Pretrained LMs use their hidden activations to encode information about inputs, which make them a natural candidate for text embeddings. As discussed in [2], this direct adaptation is blocked by the causal self-attention mechanism in decoder-only Transformer models. Autoregressive LMs are trained to predict the next token x_t given a sequence of tokens $x_0, x_1 \dots x_{t-1}$. To prevent information leaking from target tokens, the causal masking mechanism blocks any information from $x_{j \geq t}$ when computing the hidden activations of x_t . This masking strategy makes hidden activations less information-rich for earlier tokens across the input sequence length, since causal masking decontextualize these tokens from later information.

While the structural properties of state space models are extensively studied in the context of generative LMs [10, 16, 18, 19, 31], we have not seen research efforts towards understanding their hidden activations in the context of text embeddings. As discussed above, SSM-based LMs excel at computational and memory efficiency processing long-sequence inputs, which already make them desirable as text encoders. In this thesis, we further show that their state vectors are better textual representations than their Transformers-based counterparts. In signal processing and control theory literature, state vectors in state space models are designed to compress information of a linear system. This property is further exploited in designing the HiPPO[17] matrices. Despite modern pretraining of SSM-based LMs relaxed some assumptions in [17, 19], we empirically showed that state vectors in large LMs still retain this information-rich property. In addition, the final state vector after processing the entire input sequence theoretically contain contextualized information of the entire input, in contrast to the uneven contextualization in Transformer-based models. Again, directly adapting hidden activations as text embeddings in pretrained LMs have the advantage of side-stepping the need of labeled data, making it possible to train text encoders with strong generalization ability to unseen information domains.

Instruction-Following Text Encoders While existing text embedding models demonstrated strong performance in representing natural language inputs in general, they cannot adjust the embedding space to reflect application-specific requirements. Consider a database with academic papers stored in fixed-length representations. Given one selected paper, one may hope to search for other papers of the same application or same methodology. But since the encoded representations are fixed, any single similarity function can only return a fixed set of papers that cannot satisfy user-specific intentions. To the best of our knowledge, this thesis provides the first attempt to generate text embeddings that can follow instructions. INBEDDER proposed by [30] is the only potential counterpart to our study. However, they

rely on the hidden activations used to obtain prediction logits as text embeddings, which make long sequence embedding tasks impossible.

1.2 Outline

In Chapter 2, we provide a background overview of state space models. Starting from the canonical form of SSMs, we cover its core properties such as recurrence-convolution equivalence, additivity-concatenation equivalence, and popular algorithms such as the Fast Fourier Transform (FFTs) to compute SSMs efficiently in modern hardware. We will also cover main works on adapting state space parameterization in language modeling tasks, and the architectural choices made by state-of-the-arts SSM-based LMs. We will introduce the popular use cases of retrieval-augmented generation (RAG) pipeline, and the development history of different classes of text embedding models. In Chapter 3, we provide a set of individual experiments that validate our core hypothesis: state vectors in SSM-based LMs are information-rich and can be applied to text embedding tasks. A more in-depth discussion of Mamba’s model architecture is provided. In Chapter 4, we investigate the effect of instruction-prompting on text embedding performance. Evaluating the extend of instruction-following ability is a challenging and sometimes subjective task. We confirm the efficacy of instruction prompting through predicting the expected behavior of instructions and validating them on our benchmarks.

1.3 Contributions

We summarize the technical novelty and contributions of this thesis as follows:

- We provide the first attempt to train text embedding models *without* any labeled data, through the use of state vectors. Our approach improve the maximum context window of text encoder models from around 4,000 tokens to more than 50,000 tokens.

- We composed a set of experiments to validate the information-rich property of state vectors in pretrained SSM-based LMs. As discussed in Chapter 3, on our simple benchmark, state vector embeddings can even outperform some propriety text embedding models such as M2-BERT.
- State vector as embeddings are *instruction-controlled*. While past work has focused on training text encoders to follow a small, annotated set of in-domain instructions, state vector embeddings exhibit strong generalization ability to instruction types and domains. With careful selection of instruction-prompts, we can further close the performance gap of state vector embeddings and proprietary text encoders. More detailed

Chapter 2

Background and Related Work

2.1 State Space Models

The state-space representation originated from the field of signal processing to map the outputs of a linear system y_t given input signals $u_t \in \mathbb{R}$, $t \in [T]$ with the following differential equations

$$\begin{aligned}\dot{\mathbf{x}}_t &= \mathbf{A}\mathbf{x}_{t-1} + \mathbf{B}u_t \\ y_t &= \mathbf{C}\mathbf{x}_t\end{aligned}\tag{2.1}$$

where $\mathbf{x}_t$ is called the state vector. State matrix $\mathbf{A} \in \mathbb{R}^{m \times m}$ and vectors $\mathbf{B}, \mathbf{C} \in \mathbb{R}^m$ define the update rules of the underlying single-input, single-output (SISO) linear system. The general bilinear transform method allows us to discretize the above system. With any such continuous-time state space system and a discretization step size Δ , there is a corresponding discrete-time approximation

$$\begin{aligned}\mathbf{x}_t &= \bar{\mathbf{A}}\mathbf{x}_{t-1} + \bar{\mathbf{B}}u_t \\ y_t &= \mathbf{C}\mathbf{x}_t\end{aligned}\tag{2.2}$$

where $\bar{\mathbf{A}} = (\mathbf{I} - \frac{\Delta}{2}\mathbf{A})^{-1}(\mathbf{I} + \frac{\Delta}{2}\mathbf{A})$ and $\bar{\mathbf{B}} = (\mathbf{I} - \frac{\Delta}{2}\mathbf{A})^{-1}\Delta\mathbf{B}$. Since this thesis primarily focuses on discrete input space (e.g. language tokens), we will use $\mathbf{A}, \mathbf{B}, \mathbf{C}$ to represent the

discrete system parameters unless otherwise stated or inferred by context. Equations 2.2 describe a *time-invariant* model. That is, the update parameters do not depend on the time step t . One can easily extend this definition to have *time-variant* models, where $\mathbf{A}_t, \mathbf{B}_t$, and $\mathbf{C}_t$ are functions of time step t .

SSM as Recurrence. The state of the system $x_t \in \mathbb{R}^m$ at any given time is determined by its previous state x_{t-1} and current input u_t , which establishes a recursive relationship. But this recursive nature also makes it hard for RNN-like models to fully utilize modern parallel-efficient hardwares such as GPUs. In later sections, we will discuss how to improve both the computational and memory efficiency of recurrence layers.

SSM as Convolutions. Due to the discrete nature of Equations 2.2, we can roll out the output based on initial conditions. Assume $\mathbf{x}_0 = 0$, then have

$$\begin{aligned} x_1 &= \mathbf{B}u_1, \quad y_1 = \mathbf{C}\mathbf{B}u_1 \\ x_2 &= \mathbf{A}\mathbf{B}u_1 + \mathbf{B}u_2, \quad y_2 = \mathbf{C}\mathbf{A}\mathbf{B}u_1 + \mathbf{C}\mathbf{B}u_2 \\ &\dots \\ y_t &= \sum_{i=0}^{t-1} \mathbf{C}\mathbf{A}^i \mathbf{B}u_{t-i} \end{aligned}$$

Let $*$ denote the convolution operator, where for some filter h and input u of length L has $(h * u)_t = \sum_{j=0}^t h_{t-j}u_j$. Consider the following filter of length L extracted from above:

$$K = [\mathbf{C}\mathbf{B}, \mathbf{C}\mathbf{A}\mathbf{B}, \mathbf{C}\mathbf{A}^2\mathbf{B}, \mathbf{C}\mathbf{A}^3\mathbf{B}, \dots, \mathbf{C}\mathbf{A}^{L-1}\mathbf{B}] \quad (2.3)$$

Then we have time-invariant discrete state space models with input sequence $\mathbf{u} \in \mathbb{R}^T$ has output $y_t = (k * u)_t$. Therefore time-invariant SSMs have exact equivalence to both recurrence and convolutional networks.

Computation Cost of SSMs Naively computing the convolution kernel K in 2.3 incur quadratic $O(T^2)$ FLOPs, which defeats the purpose of using SSMs as alternative LM architecture. Fortunately, we can use the Fast Fourier algorithm (FFTs) to compute the point-

wise multiplication of the convolution kernels in the frequency domain. Due to the fast forward and inverse FFT algorithm, we can achieve $O(T \log T)$ FLOPs.

2.2 SSMs for Sequence Modeling

In this section we present some recent progress of using SSM backbones for language modeling.

Computational Inefficiency of Transformers While Transformer-based models such as ChatGPT exhibit outstanding reasoning ability, they suffer from quadratic computational complexity with respect to input sequence length. The cost of deploying Transformer-based LMs thus scale quadratically with the amount of external information required to perform their tasks. Consider a N -layer single-head transformer model with input sequence length L and hidden state dimension d . For each layer l , assume the input for the layer be $\mathbf{X}_l$ and the output be $\mathbf{X}_{l+1}$. Then the transformer-based update rule is given as

$$\mathbf{X}_{l+1} = \mathbf{V}_l \text{softmax} \left(\frac{\mathbf{Q}_l^\top \mathbf{K}_l}{\sqrt{d}} \right)$$

where the attention mechanism $\mathbf{Q}_l^\top \mathbf{K}_l$ requires $O(dL^2)$ floating point operations (FLOPs). Such cost at inference time underpins the high per-token access price and slow response time in the LLM-as-a-service industry; while the cost at training time imposes hard limits on LMs’ context window, restricting their downstream applications. Numerous research efforts have been focused on mitigating such inefficiency. [32, 35] proposed positional encoding techniques that allow models to extrapolate context window at inference time, processing longer sequences than seen at training time. [9, 7] reduced the memory footprint through quantization. [21] observed sparsity in pre-trained model parameters and allowed more efficient fine-tuning through low-rank approximations. These lines of work by design incur an efficiency-performance tradeoff. A more recent line of works brings down computational cost by reformulating the kernel-level implementation of the attention

mechanism [5, 6] that does not come with performance sacrifice. But these methods still assume the Transformers architecture; they optimize the coefficient instead of the scaling behavior of computational costs. Another camp of works address the scaling behavior by proposing alternative and *sub-quadratic* architectures, revisiting pre-Transformers modeling strategies such as recurrent neural networks (RNNs) and convolution neural networks (CNNs). Among these efforts, state space models (SSMs) exhibit state-of-the-arts performance when it comes to modeling capacity and inference efficiency, with as low as $O(dL)$ inference costs.

History and Development of SSMs HiPPO [17] is the foundational work on the feasibility of using SSMs to model long-range dependency inputs. In the HiPPO framework, we view state vectors x_t as an online compression of history, and asks to design state-space models that could recover the most history from fixed-length state vector x_t . Their framework is useful to arrive at closed-form solutions under well-defined measure families that assign importance to historical context. Their closed-form solutions inspired future work such as the S4 [18] architecture. S4 uses closed-form solutions found by the HiPPO framework to initialize neural networks. To pre-compute convolution kernels k_i , they decompose their transition matrices into normal and low-rank (NPLR), which allows for fast, iterative calculations.

Despite its impressive results on synthetic memory compression tasks such as the Long Range Arena (LRA) benchmark [36], S4 under-performs attention layers when it comes to language modeling. H3 [10] sets out to identify the core modeling inefficiencies in the previous SSM modeling approach. They argue that SSM operators lack explicit *compare* and *recall* mechanism compared to attention operators. The compression mechanism that HiPPO-based SSMs implement are not *data-controlled*, lacking the ability to adjust its compression and recall patterns based on input sequences. Attention score $a_t = \frac{\sum_{j=1}^t g(Qx_t, Kx_j)Vx_j}{\sum_{j=1}^t g(Qx_t, Kx_j)}$ contains the explicit $g(\cdot)$ operator that predicts the relevancy between two inputs. Based on the compression results, attention mechanism assigns weights through

$\text{softmax}(g(q, k))$ to the full, uncompressed history of input sequences. This is an extreme instance of data-controlled behavior. Motivated by this observation, H3 proposed the ShiftSSM operator to explicitly store historical tokens, and combine the outputs of the two SSM layers through a process similar to linear attention.

The aforementioned approaches require careful tuning of matrix initialization for the SSM layers to learn efficiently. In addition, the initialization methods are biased towards what researchers believe is most useful to learn textual representations. More recent work investigates methods to side-step the initialization step and directly learn a convolution layer from scratch. [11] investigated directly learning long convolution kernels without special initialization. They proposed squashing and smoothing operators to make learned filters less noisy and therefore recover full performance.

Linear Attention. While SSM architectures proposed in [10, 18] achieved high performance in sequence modeling tasks such as time-series, audio, and visual data, they still fall short in language modeling tasks compared to the Transformers family. [16, 31] borrowed insights from the linear attention mechanism to close such expressivity gap. As discussed above, the attention score is obtained via

$$a_t = \frac{\sum_{j=1}^t g(Qx_t, Kx_j) Vx_j}{\sum_{j=1}^t g(Qx_t, Kx_j)}$$

where $g(\cdot)$ is some similarity function. Linear attention assumes some nonlinear ϕ such that $g(q, k) = \phi(q)^\top \phi(k)$. Then the attention score becomes

$$a_t = \frac{\phi(Qx_t)^\top \sum_{j=1}^t \phi(Kx_j) Vx_j^\top}{\phi(Qx_t)^\top \sum_{j=1}^t \phi(Kx_j)}$$

SSM Language Model Architecture In this section, we discuss some modern architectural choices of SSM-based LMs. To close the expressivity gap between SSMs and Transformers, Mamba[16] introduced the *selective*-SSMs with time-variant parameters and gat-

ing mechanism. Previous sequence modeling works realize *time-invariant* models that allow for convolution equivalence. By contrast, Mamba drops the convolution equivalence and adopt *time-variant* SSM layers. Time-variant SSM ensures that the state vectors are updated in a *data-controlled* manner, similar to how (q, k, v) are functions of the entire input history for Transformer models. Learned parameters $\mathbf{A}, \mathbf{B}, \mathbf{C}$ are linear functions of the input sequences, which enables data-controlled compression of sequence history. Given inputs $u^{(l-1)} \in \mathbb{R}^{B \times L \times d}$, the output of a Mamba SSM layer is given by $u^{(l)} = u^{(l-1)} + o^{(l)}$ where $o^{(l)}$ is the output of element-wise product between gating mechanism $g^{(l)} = \sigma(W_g^{(l)} u^{(l-1)})$ and SSM output $s^{(l)} = \text{SSM}_{\mathbf{A}, \mathbf{B}, \mathbf{C}}(\text{Conv}(W_a h^{(l-1)}))$. Residual connections are added across each layer to preserve information flow.

Instead time-variance, Hyena[31] models achieve data-controlled update through interleaving long convolutions and element-wise multiplication of input projections, reproducing the structure of linear attentions. Given input sequence ¹ $\mathbf{u} \in \mathbb{R}^{L \times D}$, Hyena linearly project $\mathbf{u}$ into three matrices and apply short convolutions to obtain $\mathbf{x}_q, \mathbf{x}_k, \mathbf{x}_v \in \mathbb{R}^{D \times L}$. Let $\mathbf{h} \in \mathbb{R}^{N \times D \times L}$ be the learnable long convolution kernels representing a state-space parametrization. Then $y_t = \mathbf{x}_q(\mathbf{h}^k * \mathbf{x}_k(\mathbf{h}^v * \mathbf{x}_v))$. Noted that while Hyena implements the state-space parametrization in its convolution form, the parameters could be distilled into an approximately equivalent recurrence form at inference.

2.3 Retrieval-Augmented Generation

In this section we introduce the retrieval-augmented generation (RAG) pipeline and text embedding models in details. A language model is called “pretrained” if it has undergone self-supervised learning over a massive, general-purpose, and mutli-domain corpus such as the pile [12]. While pretrained generative LLMs can encode a massive amount of latent information, their knowledge is still upperbounded by what’s available in the pretraining

¹assume batch size $B = 1$ and this dimension omitted for simplicity

corpus. Therefore when it comes to domain-specific or knowledge-intensive tasks, we need to supplement LMs with external information in the format of chunks of relevant documents, a process referred to as the RAG pipeline. Adopting the terminology from [15], the vanilla RAG pipeline consists of three steps:

- **Indexing** Given a database of relevant documents to a certain task, each document is sliced into chunks of suitable lengths. These document chunks are then encoded by the text embedding model and stored in a vector database.
- **Retrieval** The same text embedding model encodes user query into a vector. Then its paired similarity function $g(\cdot)$ is used to search through the vector database for the most relevant documents. Cosine similarity is the most common similarity function used.
- **Generation** The retrieved documents and the query are combined into a coherent prompt to feed into generative LMs. The generation process could be further enhanced by prompting strategies such as multi-hop retrieval, citation generation, chain-of-thoughts, etc.

Evaluation methods of the RAG pipeline consists of more considerations and challenges than the embedding quality of text encoders. We refer to our prior work [14] for more systematic exploration on this aspect.

Contrastive Learning Past literature[39, 13] identified *alignment* and *uniformity* as two core desired behavior of text embedding models. Given a natural language input x_i , we want to pull x_i^+ that are semantically similar to x_i close to each other in the embedding space, while pushing other, semantically distant inputs further away. Mathematically, let $\mathbf{h}_i$ and $\mathbf{h}_i^+$ be the encoded representation of x_i and x_i^+ , respectively. The contrastive learning objective with a mini-batch of N pairs then is

$$l_i = -\log \frac{\exp(\text{sim}(\mathbf{h}_i, \mathbf{h}_i^+))/\tau}{\sum_{j=1}^N \exp(\text{sim}(\mathbf{h}_i, \mathbf{h}_j^+))/\tau}$$

In this context, alignment calculates the expected distance between embeddings of positively paired instances

$$l_{\text{align}} = \mathbb{E}_{(x, x^+)} [\|f(x) - f(x^+)\|^2]$$

Uniformity measures how well the embedding vectors are uniformly distributed across the embedding space

$$l_{\text{uniform}} = \log \mathbb{E}_{(x, y)} \left[e^{-2\|f(x) - f(y)\|^2} \right]$$

To achieve alignment and uniformity properties, it is crucial that labeled positive pairs (x_i, x_i^+) truthfully reflect similar semantic contents. [13, 23] also pointed out that finding “hard” in-batch negatives (x_i, x_i^-) that are structurally similar but semantically contradictory could boost contrastive learning performance.

History and Development of Text Embedding Models The first retriever models such as BM25 directly find relevant documents based on corpus statistics such as target word frequency. This sparse method is fast and efficient in finding structurally similar documents with high lexical overlap. With the advent of Transformer models such as BERT [8], the research community switched to “dense retrievers” such as DPR[22] for their learned, high-dimensional vector representations that encapsulate semantic contents. Despite attempting to be general-purpose text encoders, however, these models struggle to generalize to unseen tasks and domains [1, 34]. Data quality and coverage is the main challenge yet to overcome. Motivated by LLMs’ success in strong generalization through massive pretraining, more recent works focused on training generalizable text embedding models through efficient labeling of massive, diverse corpus. A trending line of work exploits the rich world knowledge of pretrained LLMs to generate synthetic datasets. In Gecko[23], the authors prompted pretrained LLMs to generate task categories and task-specific queries given documents retrieved from a massive internet corpus, with the hypothesis that pre-trained LMs can produce more diverse annotations than human labelers. Interestingly, [23, 27, 28] all adopted a pre-training stage on large-scale yet lower quality corpus for text

embedding models through QA datasets. The goal of this pretraining stage is to expose text embedding models to a diverse set of tasks and domains to boost their generalization ability when the higher-quality fine-tuning dataset is relatively small-scale. Text embedding models obtained from such two-stage training recipe can generalize relatively well to diverse domains. Another line of work tries to directly distill text embedding models from pretrained LLMs’ hidden activations [2]. Past work on distilling decoder-only Transformer-based LMs such as the ChatGPT and the LLaMA family. Since decoder-only model applies causal masking to avoid information leaking across the temporal dimension, [2] modified the decoder architecture to allow for bidirectional attention. Without this modification, tokens early in the sequence will not enjoy as much contextualization as later tokens. Although initialized from pretrained generative LMs, this line of work still requires fine-tuning through contrastive learning procedures discussed above.

Instruction-Following Text Embedding Models Retrieval-augmented generation can be applicable to a large variety of downstream scenarios. For some tasks, users may hope to find documents structurally or semantically similar to the ones at hand. For other tasks, users may want to search through the datastore with a selected topic. Abstractly speaking, we want the encoded representation to capture different semantic contents – and therefore positioned differently in the embedding space – given a natural language instruction that describes the desired behavior of the text embedding models. All previously discussed training strategies assume an one-to-one mapping between textual input x_i and encoded vector $\mathbf{h}_i$ given a text embedding model. But in practice we want the text embedding models to be functions of the form $f(x_i, p) \rightarrow \mathbf{h}_{i,p} \in R^d$ where p is the instruction prompt at hand. [1, 34] are among the first works to explore instruction-following text encoders. These instruction-following text encoders are trained through supervised learning with query and prompt pairs, which incurred high demand of annotated dataset with task prompts. Similar to the synthetic dataset curation process in [23], [1] used the gold document in existing benchmarks to generate natural language prompt through pretrained generative LMs. We

highlight that this supervised learning approach produce text embedding models that struggle to generalize to out-of-domain tasks absent from their annotated dataset. Despite initial success in instruction-following in a small selection of tasks, instruction-following text embedding models significantly fall short in terms of generalization ability compared to instruction-following generative LMs.

Chapter 3

Analyzing State Vectors as Embeddings

As discussed in Section 2.1, fixed-length state vectors in state-space parameterized models are designed to store compressed information of the system up to its current time step. In this chapter, we present the exploratory efforts towards understanding the properties, robustness, and limitations of applying state vectors as text embeddings. This chapter is organized by smaller, individual experiments that collectively shed insights on the spatial properties of encoded textual representations. One can view this chapter as a feasibility report of using state vectors as text encoders. We will report the following key findings:

- Using state vectors from pretrained SSM-based generative LMs already achieve embedding performance comparable to production-level encoders, while allowing for an improvement from 4,000 tokens of maximum context to more than 50,000 tokens.
- Swapping state vectors between two prompts can recover the factual recall from one prompt to another, indicating that compressed information used for generation is stored in state vectors.
- Semantic perturbations show that state vectors are more sensitive to linguistic structure (as opposed to semantic concepts) than production-level text encoders.

3.1 Baseline Embedding Performance

To validate our core hypothesis that state vectors are information-rich representations of the input sequence, we apply state vectors off-the-shelf as text encoders and compare its performance to production-level encoders. To our surprise, this simple baseline achieves embedding performance comparable to specialized encoder models on a simple benchmark. We compare two classes of SSM models. As discussed in Section 2, *hybridation* between SSM layers and attention layers can sometimes lead to robust downstream performance. For the *hybrid* model class, we choose StripedHyena-7b as a representative model. For the *non-hybrid* model class, we choose Mamba-2.8b.

3.1.1 Textual Encoder from Pretrained SSMs

Design Assume a pretrained generative language model $\mathcal{M}$ with N SSM layers. Each of these N layers explicitly or implicitly contain state parameters $\{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$ used for $\text{SSM}_{\mathbf{A}, \mathbf{B}, \mathbf{C}}(u)$. Depending on whether the SSM layer is time-variant or time-invariant, learned state parameters could be either fixed-length matrices or tensors that scale with sequence length T . State vector $\mathbf{x}_t \in \mathbb{C}$ is uniquely defined by the state parameters and a discretization delta Δ given a sequence of inputs u_t . Unless otherwise stated, we use the default discretization scheme of pretrained generative LMs. We allow inputs to $\text{SSM}_{\mathbf{A}, \mathbf{B}, \mathbf{C}}^{(l)}(\cdot)$ to undergo an arbitrary transformation $g(\cdot)$ of the inputs of layer l described by the pretrained base-model architecture. We will choose the flattened state vector $x_T^{(l)}$ as the output of the textual encoder model given input sequence length T and a hyperparameter l .

Modeling Capacity Modern deep SSM-based LMs have many architectural complexities on top of the state space representation itself, such as short convolutions, gating, residual connections, hybridizations, etc. By dropping these complexities when building the textual encoder model, we are effectively isolating the amount of contextual information accessed by the state space model in a deep layer. As discussed later, the most significant

information loss comes from the *residual stream*. Similar to Transformer-based models, SSM-based models maintain a stream of residual connections to help information flow across layers.

Computational Efficiency Our approach requires the pretrained model to materialize at least the final state vector $x_T^{(l)}$ at selected layer index l . For SSM kernels that either maintain the convolutional form of the state space parametrization or assemble output vectors in parallel, materializing the state vector incurs additional memory and computational costs.

3.1.2 Evaluation Benchmark

The Massive Text Embedding Benchmark (MTEB)[26] is among the most popular benchmarks for textual embedding quality. MTEB is a collection of 7 textual embedding task types that span across 65 datasets. However, the MTEB benchmark requires careful tuning of instructions [2] and its sheer size makes it not suited for exploring design choices. Therefore we curated a custom textual embedding benchmark adapted from the Human-in-the-loop Attributable Generative Retrieval for Information-seeking Dataset (HAGRID). We name our custom dataset HagridEval.

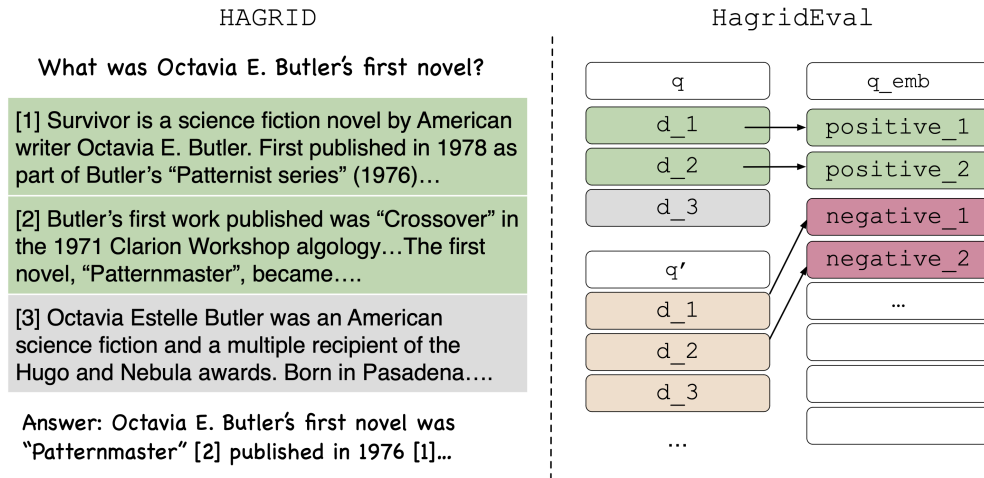


Figure 3.1: Illustration of HAGRID data instance (left). HagridEval creation process (right).

Dataset Creation. As shown in Figure 3.1, each data instance from the original HAGRID dataset contains a query q_i , a set of relevant documents $d_{i,1} \dots d_{i,n}$, and a human-written answer with attributions. We label documents that are referenced (attributed) by the answer as “positive” documents to the query. Since the supporting documents in the original HAGRID dataset are composed in an automatic and noisy manner, this additional filtering step serves to guarantee the relevancy of selected document embeddings. Given a query q_i , we randomly select indexes $j \neq i$ and use their relevant documents as “negative” documents to q_j . We remark that this selection process is similar to in-batch negatives in the contrastive learning framework. A data instance in HagridEval contains a query q , a list of documents $[d_1 \dots d_n]$, and a list of index $[i_1, \dots i_k]$ indicating the positive documents, where $k \ll n$. In our current configuration, $n = 15$ and $k \leq 3$. Again, positive documents refer to documents that are cited from the human-written answer to the query, while negative documents are analogous to “in-batch negatives” from other queries.

Evaluation Metrics We use three metrics to evaluate the embedding quality of a textual encoder model. Given an encoder model $\mathcal{M}$ and a similarity function $g(\cdot)$, we sort documents $[d_1, d_2, \dots d_n]$ to query q by their similarity score $s_i = g(\mathcal{M}(q), \mathcal{M}(d_i))$. Let k be the number of “positive” documents in the instance. PRECISION measures the number of “positive” instances in the top- k of sorted documents. PASS@M is a boolean metric that returns 1 if all “positive” documents are in the top- m sorted documents. We have $n > m > k$.

Similarity Function State vectors $\mathbf{x}_t \in \mathbb{C}$ could take either real or complex values depending on the base-model architecture. Therefore our default similarity function $g(\cdot)$ is a modification of the cosine similarity function in $\mathbb{R}$ through the Hermitian inner product:

$$g(u, v) = \frac{\overline{u}^\top v}{\|u\| \|v\|}$$

As discussed in more detail in Chapter 4, state vectors are not designed to pull semantically similar information closer in the embedding space; rather, since they are taken directly from generative LMs, their positioning in the embedding space is more correlated with future sequence prediction. This finding necessitates more thorough examination of alternative similarity functions and alternative prompting, which will be discussed in Chapter 4.

3.1.3 Results and Discussion

Comparison to Production-Level Models We choose the best-performing layer index l for each SSM-based model and evaluate on the HagridEval benchmark. Three production-level text encoders were evaluated on HagridEval for comparison, as reported in Table 3.1. Without any fine-tuning, pretrained generative Mamba can produce text embeddings that already outperform M2-BERT. Another important characteristic of text encoders is context window – the maximum sequence length processable. We highlight that none of the text encoders referenced by this thesis¹ can process documents longer than 4,000 tokens, which translates to 3,500 words, shorter than a college-standard essay. Meanwhile, SSM-based generative LMs are trained with sequence lengths at million-token scales². Thus, state vectors should be able to encode extra-long documents into fixed-size embeddings, at the cost of slightly higher memory consumption.

Isolating Performance by Layer Index Recall that our baseline approach allows for hyperparameter search over l , the layer index from which state vectors are obtained. In Figure 3.2 and Figure 3.3 we report the efficacy of state vectors across layer dimensions³. A major takeaway from this experiment is that not all layers contain information-rich state vectors. Our hypothesis is that, due to the existence of skip connections, some layers can afford to

¹Except for the MISTRAL encoder, which does not report maximum context window.

²Please refer to Github repository at <https://github.com/jzhang38/LongMamba> for detailed analysis of Mamba maximum context length.

³In this thesis, larger layer index universally means closer to the final output layer.

	PRECISION	PASS@5	PASS@10	Context	Dimension
MAMBA	0.96	0.99	0.99	50,000+	
STRIPEDHYENA	0.52	0.8	0.93	50,000+	
M2-BERT	0.74	0.91	0.99	8162	768
VOYAGE-2	1.0	1.0	1.0	4,000	1024
MISTRAL	1.0	1.0	1.0	-	1024

Table 3.1: Baseline embedding performance on the HagridEval dataset. Mamba and StripedHyena models produce embeddings through average pooling of their state vectors across the layer dimension.

process a subset of information present in the input sequence. This phenomenon is correlated with state collapse, present in the training process of many SSM-based LMs such as [16, 25, 31], where certain channels or certain dimensions in a channel shrink to constant with across the sequence length dimension.

Visualizing State Collapse To further investigate the behavior of state vectors across different layers, we visualized the dynamics of different layers. As partially⁴ shown in Figure 3.4, there is a high correlation between state vector embedding performance and state collapse. We note that since Mamba layers are *time-variant*, the collapsing behavior is data-controlled and therefore cannot be pruned without careful fine-tuning. We defer further investigation of state collapse and its behavior to future studies.

Limitations We report the maximum sequence length for generation as context window for SSM-based LMs in Table 3.1. Due to the lack of extra-long sequence text embedding benchmarks, the embedding performance of state vectors at compressing extra-long input sequences remains unexplored. It is possible that embedding performance could degrade with longer sequences. Since state vectors are trained to store information useful for next-token prediction, they could be very sensitive to the semantic instructions of the last few tokens in a sequence. Future work could focus on either creating a smaller-scale but long-sequence natural language embedding benchmark, or curating extra-long synthetic embedding tasks similar to the Long Range Arena [36] benchmark.

⁴A complete visualization can be found in our Github repo due to file size constraints.

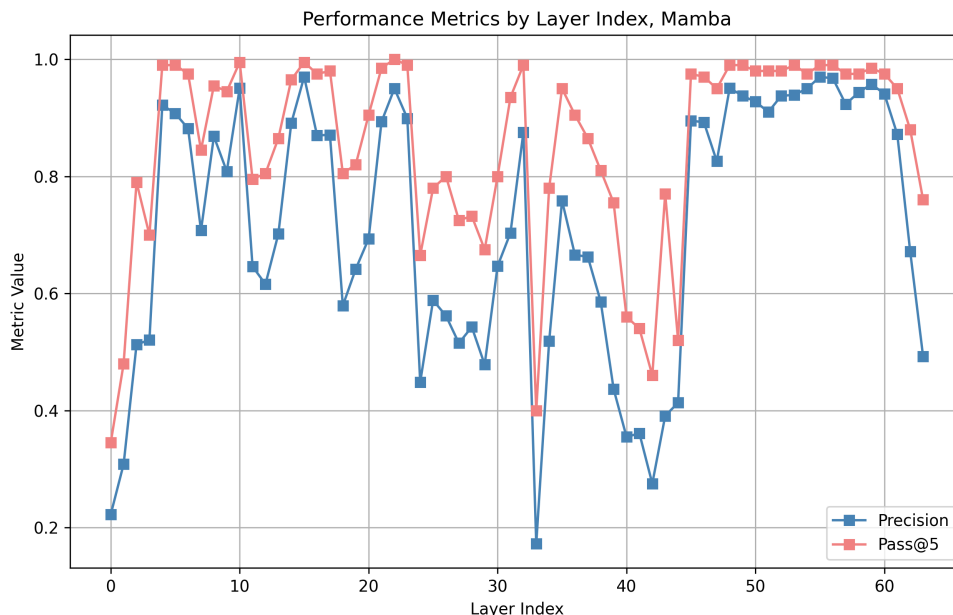


Figure 3.2: HagridEval embedding performance by layer, with Mamba-2.8b as base LM.

3.2 Locating Factual Knowledge in State Vectors

As discussed in Chapter 1, a major advantage of using state vectors as text embeddings is the possibility of initializing generative LMs with packed external information. In this section, we investigate the mechanism of factual recall in state vectors. At the start of generation, current SSM-based LMs initialize their state to the zero vector. We beg the question of alternative initialization schemes to fully utilize the recurrent nature of SSMs. To understand the gap and limitations in directly initializing LMs with cached state vectors, we use activation patching[38] to localize information in Mamba’s activations.

3.2.1 Activation Patching

Factual recall or association refers to a language model’s ability to produce the correct object o given a relation description r and subject s . Activation patching is an analytical method successfully applied to other neural architectures such as the Transformer models [33, 38], which perturbs input sequences to locate the hidden activations that significantly

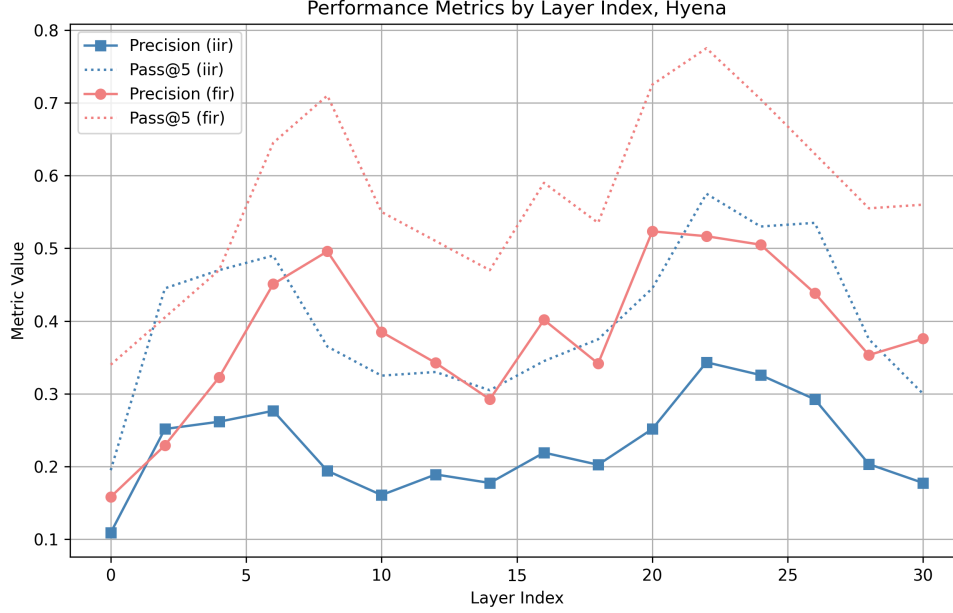


Figure 3.3: StripedHyena-7B. Orange lines represent the embedding score of short convolution filters, while blue lines represent SSM state vectors.

influence the model’s ability to perform factual associations. Figure 3.5 illustrates the patching process on a simplified neural architecture.

- **Clean Run** The clean run starts with a fact triplet of subject, relation, object denoted as (s, r, o) that the language model already knows. (s, r) is passed to the model to obtain computed activations $\mathcal{S}_{\text{clean}}$.
- **Corrupt Run** There is a variety of methods to corrupt the clean fact triplet (s, r, o) . The most data-efficient method is to replace the subject tokens with Gaussian noise. But [42] showed that Gaussian noise could incur non-representative behavior due to out-of-domain hidden states. Therefore we proceed with swapping target object o with alternative, natural language object o' . We then sample another fact triplet (s', r, o') that the model does not necessarily know. Noted that relation r needs to maintain constant. For instance, in Figure 3.5, $r = \text{“the headquarters of X is in Y”}$ remains constant through different runs. Denote the computed activations corre-

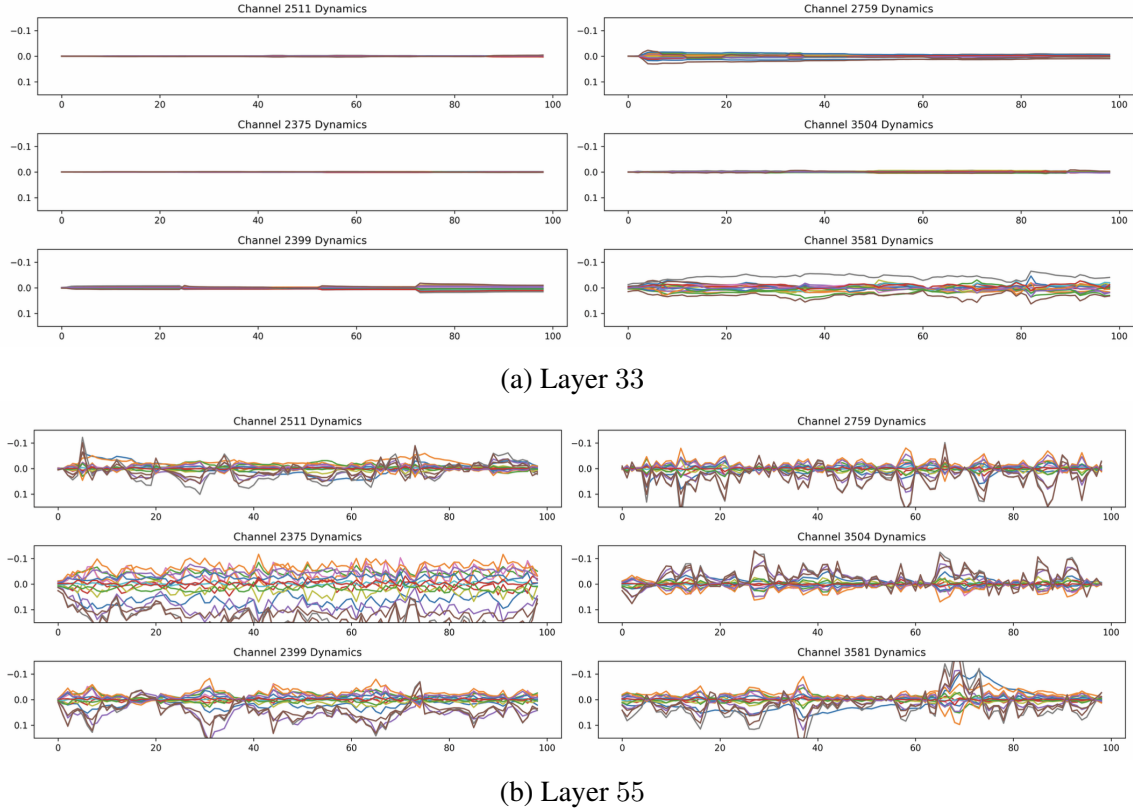


Figure 3.4: Visualization of state collapse behavior in Mamba across sequence length. Channels are randomly sampled.

sponding to (s', r, o') as $\mathcal{S}_{\text{corrupt}}$.

- **Patched Run** In the patched run, we feed the same prompt from corrupted run to the model. We breakpoint inference at $\mathcal{S}$, and replace the hidden activations $\mathcal{S}_{\text{corrupt}}$ with cached $\mathcal{S}_{\text{clean}}$ and continue inferencing.

Metrics The *indirect effect* metric of hidden activation set $\mathcal{S}$ is given by

$$\text{IE} = \frac{\hat{p} - p_0}{p - p_0}$$

where $p = P(o|s, r, \mathcal{S}_{\text{clean}})$, $p_0 = P(o|s', r, \mathcal{S}_{\text{corrupt}})$, and $\hat{p} = P(o|s', r, \mathcal{S}_{\text{clean}})$. IE can be interpreted as the ratio of factual recall recovery. In this setup, model is said to *know* a fact triplet (s, r, o) if the probability of $P(o|s, r)$ is higher than all alternative subjects $P(o|s', r)$. Higher IE corresponds to hidden activations $\mathcal{S}$ playing more important roles

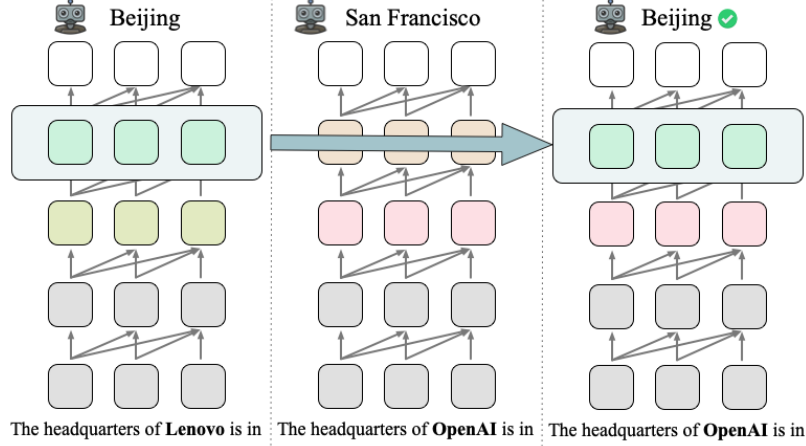


Figure 3.5: Simplified illustration of activation patching, with clean run (s, r) (left), corrupted run (s, r') (middle), and patched run (s, r') (right). Highlighted sections refer to the hidden activations cached and restored. In the ideal case, patched LM should be able to perform factual recall associated with (s, r) and output *Beijing*.

at factual association with respect to relation r . Negative IE score is only possible when $\hat{p} < p_0$. That is, patching hidden activations with clean cache $\mathcal{R}_{\text{clean}}$ reduce the probability of target subject s .

3.2.2 Experiment Setup

Dataset We use the RELATIONS dataset[20] for the patching experiment. The dataset contains short triplets across factual, commonsense, linguistic, and bias domains, each containing 5 to 20 simple relations. We filter the RELATIONS dataset based to only contain facts that the mamba-2.8B model already knows. Since our goal is to test the efficacy of state vectors at restoring factual knowledge, target hidden activations are $\mathcal{S} = \{x_t^{(l)}\}_{t \in [T]}$ with layer index hyperparameter l . Our patching strategy only replace the state vectors with cached vectors, leaving state matrices $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$ and input signals untouched.

Modeling Mamba [16] models are *non-hybrid* and therefore most suited for the activation patching experiment. A mathematical formulation of Mamba’s model architecture is provided in Section 2.2. Figure 3.6 further illustrate the layout of a state-space parameterized layer of Mamba models. We highlight that (patched) state vector $x^{(l)}$ undergo pairwise

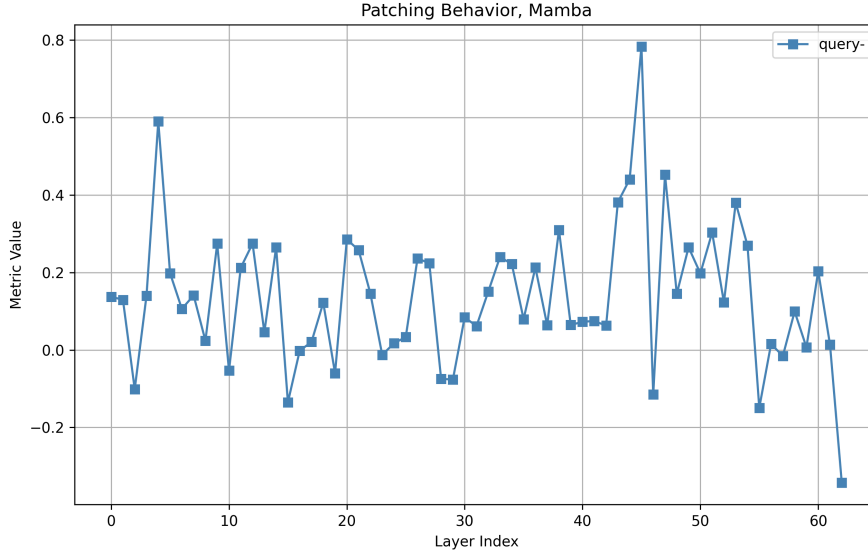


Figure 3.7: Activation patching results in Mamba-2.8b

3.3 Semantic and Lexical Perturbation

The previous patching experiment explore the efficacy of restoring factual association by state vector splicing. It could be viewed as another set of validation experiments on the information-rich property of state vectors and further confirm its potentials as embedding vectors. As discussed in Section 2.3, text embedding models are trained to pull together semantically similar texts in the embedding space. The practical definition of “semantically similar” is determined by both positive pairs and hard negative pairs curated during the annotation process. Given the *alignment* and *uniformity* properties of text embedding models trained through variants of the contrastive learning objective, cosine similarity becomes a natural choice for similarity measurement in the RAG pipeline. However, state vectors are trained to store relevant information for next-token prediction. They do not necessarily position semantically similar contexts closer in the higher-dimensional space, and may be sensitive to other linguistic features such as sentence completeness. In this section, we further explore the impact of semantic perturbations on state vectors as text embeddings. Through a collection of datasets, we can evaluate (1) what linguistic features are reflected

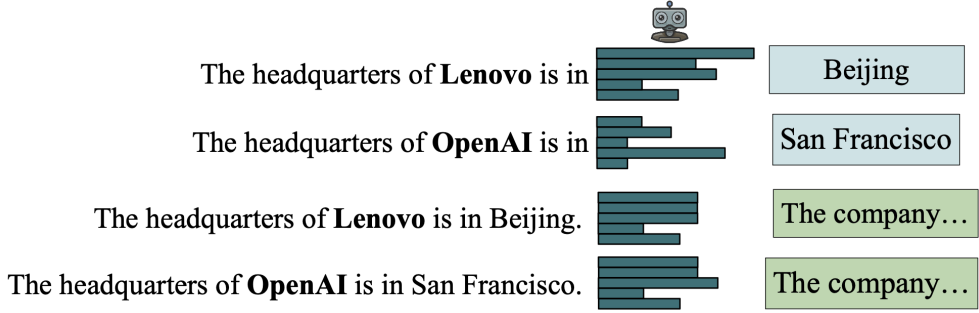


Figure 3.8: Illustration of sentence completeness and its impact to expected next token, measured by mamba-2.8b

in state vectors, and (2) how linguistic structures and semantic meaning are weighted.

3.3.1 Sensitivity to Sentence Completeness

Motivation SSM-based generative LMs are *auto-regressive* – they are trained to predict the next token given an input prompt. Intuitively, the expected next token of a complete sentence has very different distribution compared to the next token of an incomplete sentence, as illustrated in Figure 3.8. Given the drastic difference in expected next token distribution, the state vectors in generative LMs may also be over-sensitive to the sentence structural properties such as completeness. Sensitivity to sentence completeness is a major side-effect of auto-regressive training of generative LMs, which should be considered an out-of-domain field for text encoders. This experiment is partially designed to distill the impact of autoregressive training in using state vectors as text encoders.

Dataset Construction Directly taking the filtered RELATIONS dataset, we construct the SENTENCECOMPLETENESS benchmark with two subsets. The first subset tests a text encoder’s sensitivity to semantic content in *incomplete* sentences with only subject s and relation r . The second subset tests the text encoder’s sensitivity to semantic content in *complete* sentences with (s, r, o) . As discussed in Section 3.2, we filtered out factual rela-

tions that the model does not know, measured by its log-probability over the object given (s, r) pair. For each subset in SENTENCECOMPLETENESS, we take the average of similarity score between randomly sampled q, q' without replacement as the subset similarity score.

3.3.2 Sensitivity to Semantic and Structural Similarity

Motivation As discussed in Section 3.3.1, incomplete sentences are considered an out-of-domain since most labeled dataset for text embedding supervised learning contain complete sentences exclusively. Therefore comparing sensitivity to sentence completeness sheds light on the impact of auto-regressive training but fall short in describing the qualitative behavioral difference between state and production-level text encoders. In this section, we further construct a dataset with controlled structural similarity between *complete* sentences.

Dataset Construction We remark that current, popular evaluation suits for text encoders do not describe the encoders’ sensitivity to linguistic aspects of inputs. But such sensitivity exploration is crucial for us to understand the gap between state vectors and desirable text embeddings. Neither is there any existing dataset that could be easily adapted to contain (1) structurally similar pairs with different semantic concepts, and (2) semantically related pairs with drastically different sentence structure. For instance, (s, r, o) pairs in the RELATIONS dataset can serve as the former, while (query, relevant document) pairs in HAGRIDEVAL could serve the latter. But no existing dataset can be adapted to contain a triplet of aforementioned properties. Motivated by this observation, we constructed the STRUCTURE dataset with 2,000 instances by prompting ChatGPT with manual filtering. With each query q from the HAGRID dataset, we prompt ChatGPT to produce three structurally similar but semantically different queries $\{q'\}$. Following the prompting recommendations from our past work [41], ChatGPT is given three in-context examples as system prompts.

3.3.3 Results and Discussion

Evaluation results of aforementioned tasks are reported in 3.2.

State vectors are more sensitive to sentence completeness As highlighted in the COM-

	STATEVECTOR		M2-BERT		VOYAGE-2	
	STRUCTURE					
	<i>query-</i>	<i>doc+</i>	<i>query-</i>	<i>doc+</i>	<i>query-</i>	<i>doc+</i>
	0.829	0.630	0.569	0.608	0.830	0.813
	COMPLETENESS					
	+	-	+	-	+	-
Fruit Color	0.939	0.471	0.716	<u>0.345</u>	0.844	0.426
Object Class	0.953	0.478	0.945	0.471	0.899	<u>0.447</u>
Occupation	0.708	0.357	0.149	<u>0.088</u>	0.814	0.414
Degree / Gender	0.940	0.469	0.801	<u>0.401</u>	0.870	0.435

Table 3.2: Semantic perturbation results. See Section 3.3.2 for explanation of the STRUCTURE benchmark and Section 3.3.1 for explanation of the COMPLETENESS benchmark. Pretrained Mamba-2.8b with average pooling is used for STATEVECTOR statistics.

PLETENESS section in Table 3.2, state vectors from auto-regressive LMs are indeed more sensitive to sentence completeness than specialized text encoders at the best performing encoder layer $l = 55$. But a natural question is whether this sensitivity remains uniform across all layers in SSM-based LMs. We report the change of evaluation result on the aggregated STRUCTURE dataset.

Structural similarity dominates semantic similarity for state vectors As highlighted in the STRUCTURE section in Table 3.2, structurally similar but semantically distant inputs are close to each other in the embedding space than semantically relevant but structurally different pairs. Surprisingly, even production-level text encoders could suffer from this phenomenon. A key difference between specialized text encoders and state vectors is that structure and semantics are weighted similarly in determining the text embedding: for encoder models such as M2-BERT and VOYAGE-2, the impact of structural and semantic similarity yields similar cosine similarity. Following previous discussion, we visualized the impact of structural and semantic distance across layer index reported in Figure 3.9. While structural similarity consistently play an important role in determining the text embeddings,

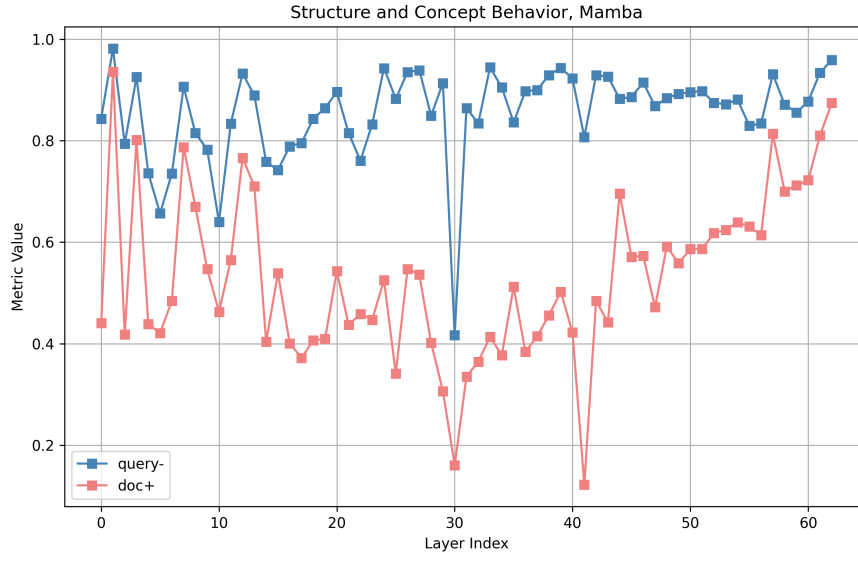


Figure 3.9: Structural and semantic similarity weighing. All data sampled from the RELATIONS dataset. Given a target query q_i , QUERY- refers to a query q_j that has similar lexical structure such as word count and grammar, but with *different* topics. DOC+ on the other hand refers to the on-topic document corresponding to q_i , but is usually much longer and structurally different from q_i .

the importance of semantic similarity surprisingly depend heavily on layer index.

Chapter 4

Aligning Embedding Space through Instructions

Chapter 3 provides two main insights. First, state vectors are information-rich and can be applied off-the-shelf as text embedding models. Second, in contrast to text embedding models trained through contrastive learning, state vectors are *not* explicitly trained to maintain alignment and uniformity of the corresponding embedding space. Semantically distinct inputs could be positioned closely in the state vector embedding space if the model predicts similar future sequences for both inputs. Exploring alternative similarity functions is a potential direction to solve this limitation. But we highlight that there is no off-the-shelf alternative, since the behavior of the encoded representation is subject to next-token prediction and therefore largely intractable. Meanwhile, a learned similarity function restricts its corresponding text encoders to a small subset of task domains specified by its training dataset. Therefore, adapting state vector encoders to new domains requires dataset collection and careful tuning, which contradicts the purpose of our label-free text embedding methodology. In this chapter, we turn the aforementioned challenge into a unique advantage of state vectors through instruction prompting. This chapter is composed of various methods to distill the effect of natural language instruction. We will present the following

key findings:

- Following [4, 24], we visualize the effect of modifying inputs with natural language instructions using 2D t-SNE[37] plots. Visualization and evaluation results confirm that instruction-prompting can reshape the state vector representation space guided by desired concepts, topics, etc.
- We compare the effect of instruction-prompting on LMs with and without instruction-tuning.

4.1 Introduction

While state vectors are information-rich, their positioning in the high-dimensional embedding space is highly sensitive to the lexical structure and future trajectories of natural language inputs. Instead of learning similarity mapping in this highly intractable and sensitive embedding space, we instead explore methods to control and reshape the embedding mapping behavior subject to task-specific goals.

4.1.1 Background

t-SNE (t-Distributed Stochastic Neighbor Embedding) is a powerful, non-linear technique for dimensionality reduction. It learns to translates the high-level Euclidean distances between data points into conditional probabilities that represent similarities. The similarity of datapoint x_j to x_i is the conditional probability $p_{j|i}$ that x_i would have x_j as its neighbor, if neighbors were picked in proportion to their probability density under a Gaussian centered at x_i .

Next-Token Prediction Despite repeated references to this concept, we have not discussed the implications of next-token prediction training of generative LMs up until this point. As the name suggests, this method involves training a language model to predict the next

token in a sequence given tokens that precede it. Given tokens $(x_1, x_2, \dots, x_{T-1})$, the model predicts the next token x_T and receives loss $L(\theta) = \prod_{t=1}^T P(x_t | x_1, x_2, \dots, x_{t-1}, \theta)$. Hypothetically, if a model can flawlessly complete any sentence, it effectively embodies the capabilities of, say, the most knowledgeable and unbiased person – encapsulating the essence of general intelligence. But this next-token prediction also means that pretrained LMs such as Mamba used above do *not* have a built-in purpose. Given a natural language input, they will try to compress information to predict the next token instead of summarizing the key semantic contents of the passage.

4.1.2 Problem Definition

Assume a pretrained language model $\mathcal{M}$ trained through next-token prediction objective that can generate embedding $\mathbf{h}_T$ given natural language inputs $\{x_t\}_{t \in [T]}$. Assume a textual embedding task with a particular focus such as general summarization, topic, sentence structure, etc. with a natural language prompt p_{focus} . Our goal is to find p_{focus} such that $\mathcal{M}(x_1, x_2, \dots, x_T, p_{\text{focus}})$ yields $\mathbf{h}_T$ in an embedding space $\mathcal{S}$ in which the Euclidean distance between $\mathbf{h}$ captures the semantic relevance of inputs subject to the given focus. Figure 4.1 illustrates the core idea of instruction-controlled representation. Since $\mathcal{M}$ is trained for next-token prediction, we prompt $\mathcal{M}$ to generate what we hope to encode. Table 4.1 provides some instruction examples. Again, these instructions are designed to prompt our generative model to store relevant semantic content in its hidden activations, which are then used for embedding.

Focus	Instruction Postfix
Summary	Below is a summarization of the above document:
Sentence type	Between {}, {}, and {}, the above text falls in the category of
Custom focus	What are the main arguments and information provided above about {}?

Table 4.1: Example of zero-shot instruction prompting.

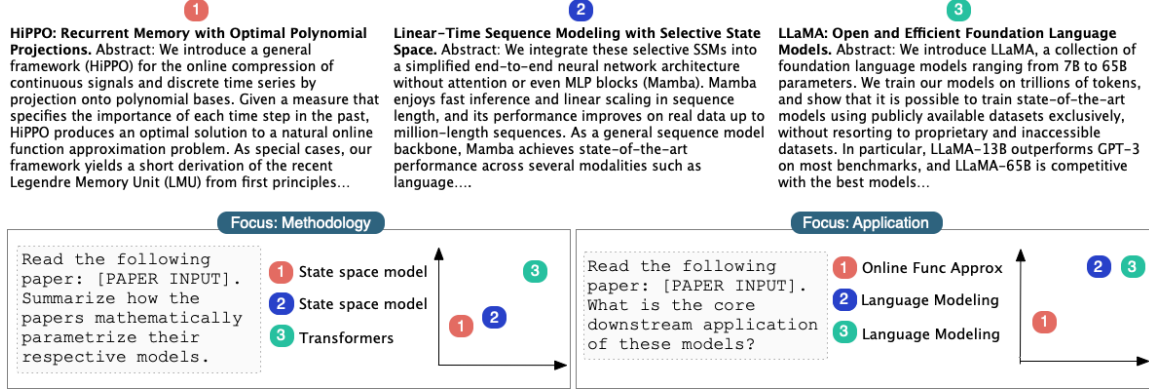


Figure 4.1: Illustration of Instruction-Controlled Embedding. Here each document represents a paper to encode. Each box represent a different embedding space induced by its corresponding focus. In the ideal scenario, the Euclidean distance of vectors in the embedding space reflect the relevancy of documents with respect to given *focus*.

4.2 Instruction Prompting Setup

Evaluating the instruction-following ability of generative LMs is already a loosely-defined and challenging task. As discussed in our prior work [41], there is a lack of annotated datasets on the desired behavior of instruction-following generative LMs. While LLM-annotators could be used for natural language instruction-following, there is no automated and scalable way to evaluate the extent of instruction-control over textual embeddings. To the best of our knowledge, there also are not any annotated dataset to measure the instruction-following ability of textual embedding models. Therefore, we choose to *implicitly* evaluate the influence of instruction. Reusing the same benchmarks discussed above, we aim to predict the embedding behavior given categories of instructions, and validate our hypothesis through aggregated embedding performance. Similar to the Chapter 3, we use Mamba-2.8b *without* instruction-tuning as our base LM.

HagridEval Mark Each data instance in the Hagrid dataset contains a query q , a list of $N = 15$ documents $(d_1, d_2, \dots, d_N)$ where $k \leq 3$ of them are relevant to q . A more detailed description of its curation process can be found at Chapter 3. We explore two types of instructions in this setup. The first category aims to map texts to their encoded representations based on their semantic topic. The second aims to map through sentence type.

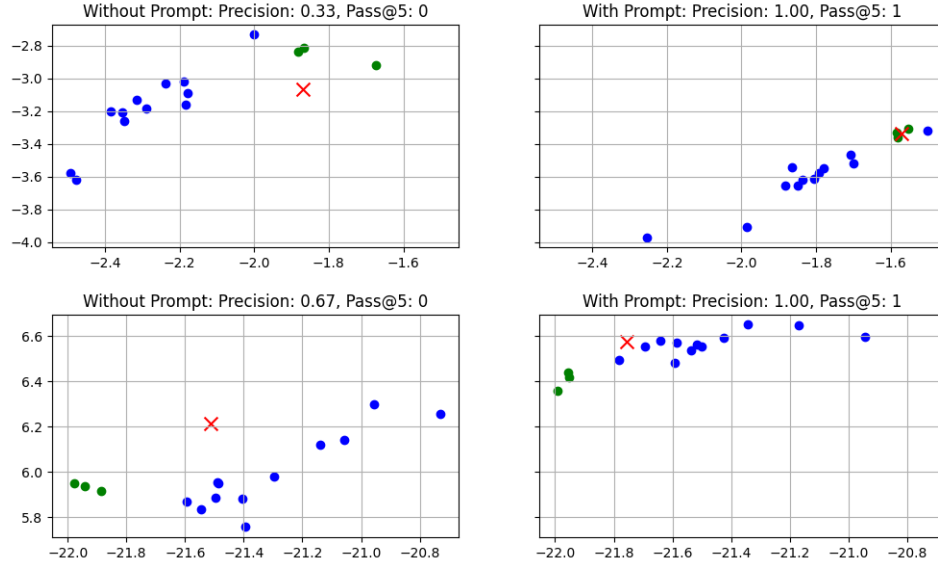


Figure 4.2: t-NSE visualization of instruction-controlled representation, with focus on **semantic topic**. Respective embedding performance is reported. Instruction prompt: In one word, the above text is about the topic of `{}`. In this figure and Figure 4.3, the red mark refers to the projected embedding of the *query*. Green mark refers to the *gold* documents, while blue mark refers to the rest.

Note that the first category is useful for relevant document retrieval. We selected instances from HAGRIDEVAL where the Mamba-2.8b model did not achieve a perfect score in the previous chapter for visualization. We compute the average pooling of state vectors across layers as the encoded representation.

Relations Benchmark We want to revisit the experiments conducted in Table 3.2. If state vector representations exhibit instruction-following ability, we should be able to reverse the results obtained in both the STRUCTURE and COMPLETENESS subset. As shown in Table 3.2, state vectors are highly sensitive to sentence lexical statistics and completeness. In contrast, embeddings trained via supervised learning¹ shown smaller embedding gap and therefore higher robustness to the “appearance” of inputs. The goal of this section is to reduce the embedding similarity gap between *query*- and *doc*+ through instruction prompting.

Visualization Method Following the common practice discussed in [4, 24], we apply PCA

¹The training method of VOYAGE is not publicly available.

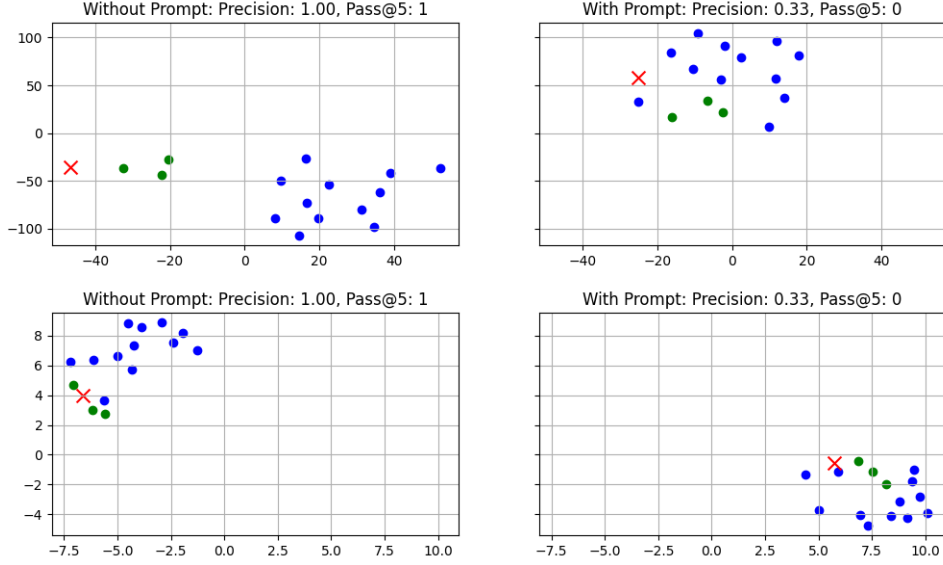


Figure 4.3: t-SNE visualization of instruction-controlled representation, with focus on **sen-
tence types**. Respective embedding performance is reported. Instruction prompt: The
above text is a form of {}.

dimensionality reduction followed by t-SNE to visualize high-dimensional state vectors in 2D. Each data instance is fitted by a separate t-SNE model. This leads to t-SNE models having a very small scale of dataset ($N = 32$ for two prompting configurations), therefore we choose the perplexity hyperparameter to be around 10. Since t-SNE projections from $d = 5120$ to $d = 2$ with less than 40 datapoints can sometimes be noisy, we report PRECISION and PASS@5 metrics for embedding performance reference.

4.3 Results and Discussion

In this section, we show that instruction-prompting can change the relative positioning of state vector embeddings, so that Euclidean distance measured by cosine similarity is sufficient to measure the relevancy between two inputs under a given focus.

HagridEval Figure 4.2 and 4.3 visualizes the impact of instruction prompts with focus on *semantic topic* and *sentence types* respectively. While t-SNE visualization cannot fully capture the high-dimensional structure of embeddings, we see a general trend that, for

No Prompt		Semantic Topic		Sentence Structure	
<i>query-</i>	<i>doc+</i>	<i>query-</i>	<i>doc+</i>	<i>query-</i>	<i>doc+</i>
0.829	0.630	0.848	0.876	0.952	0.875

Table 4.2: Semantic perturbation results with prompting. See Section 3.3.2 for explanation of the STRUCTURE benchmark and Section 3.3.1 for explanation of the COMPLETENESS benchmark.

instruction prompts with focus on *semantic topic*, encoded vectors of query q and gold documents are pulled closer to each other. In contrast, instruction with focus on *sentence types* tend pull q and gold documents further apart, while pulling gold documents closer to other documents of the same sentence types. We further confirm our core hypothesis in the last section that, with instruction prompting focused on semantic topics, state vectors can summarize inputs better and therefore achieve higher embedding score than their no-prompting counterparts. We report the embedding precision of state vectors by layer, reported in Figure 4.4. As shown in the figure, adding instruction prompts with focus on semantic topic boosts embedding performance significantly, across all layers without state collapse. This confirms that, as hidden activations of generative LMs, state vectors require instruction prompting to release their full potential as text embeddings.

STRUCTURE Benchmark Table 4.2 reports the main results of feeding instruction prompts to the STRUCTURE benchmark with focus on topic similarity. First, the results confirm our hypothesis that instructing state vectors to focus on semantic topics can mitigate the impact of document structure on embedding similarity. But the similarity score between gold query q and a semantically different yet structurally similar query $q-$ also increased even when asked to focus on semantic topic. Vice versa, the similarity score between gold query q and gold document $d+$ also increased, despite when asked to focus on sentence structure. Our hypothesis is that instruction prompts effectively reduced the probability distribution of future trajectories of input sequences. Therefore the base similarity score increased.

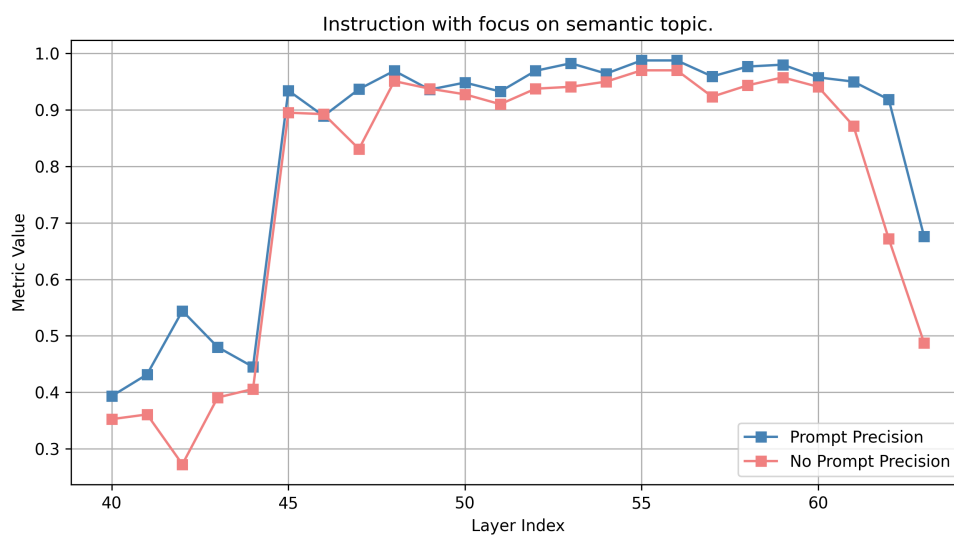


Figure 4.4: Instruction prompting effect on HagridEval, by layer.

Chapter 5

Conclusion

5.1 Overview

Text embedding models are ubiquitous in information system applications. But the contrastive-learning approach, while effective, suffers from two inherent limitations. First, contrastive-learning requires annotated labels. This drastically limits the scale of training corpuses. While generative LMs can learn from virtually all available texts from the Internet, Text embedding models using contrastive-learning objectives can only learn from a small subset of them. Second, such training objectives can only yield general-purpose textual representations. Contemporary textual encoders may fail at downstream applications that need to encode inputs with targeted concepts, domains, or other properties.

State-space model (SSMs) emerge as a powerful alternative architecture to the Transformer family for its low computational and memory costs. But we highlight that its structural properties remain largely unexplored. Through careful distillation of its state vectors, Mamba-2.8b pretrained on the Pile can directly serve as a text embedding model. They exhibit **strong domain generalization** ability as text embeddings, while able to alter their embedding space through **instruction-control**. These abilities, along with an long context window, can serve to address the aforementioned bottleneck of text embedding models.

5.2 Limitations and Future Work

We want to begin by highlighting that the core spirit of this work is training text embedding models through self-supervised learning. SSM state vector is but one of the many potential solutions to this problem. During this thesis, we tried other approaches to autoregressively train text embedding models, but none seemed to work as well as off-the-shelf state vectors. Future work can explore this direction in more depth. The general goal should be forcing certain fixed-length activations to be information-rich; all information from earlier layers of the model should flow through the chosen activations for this to happen. The existence of a residual connection stream is likely the reason why state vectors still exhibit embedding quality gaps to state-of-the-art models. We also highlight that *instruction-following ability* is not inherent to SSM-based LMs. Instead, models trained with next-token prediction loss should all exhibit this ability when applied to text embedding models. Future work should continue exploring ways to either improve the information-richness of state vectors through a different loss function, or explore other model architectures and training recipes to train text encoders without labeled data. As repeatedly mentioned before, state vectors are responsible to compress information for next-token generation in generative LMs. Therefore, its embedding capacity should remain constant within the training context window of the LM, in our case being 40,000. However, evaluating the quality of text embeddings across the document length dimension is not in the scope of this project. Future work can easily evaluate this aspect of state vector embeddings given sufficient compute power. Finally, we want to remark that instruction prompting can be highly unstable. There is no straightforward and systematic method to design instruction prompts given an embedding task. The trial-and-error approach adopted by this thesis is therefore not suitable for large-scale benchmarks like MTEB. While instruction-tuning is a well-studied and active research field for generative LMs, its equivalence in text embedding models is non-existent. Future work on this direction should focus on creating evaluation benchmarks that makes more systematic studies on instruction-following text embedding models feasible.

Bibliography

- [1] A. Asai, T. Schick, P. Lewis, X. Chen, G. Izacard, S. Riedel, H. Hajishirzi, and W.-t. Yih. Task-aware retrieval with instructions. *arXiv preprint arXiv:2211.09260*, 2022.
- [2] P. BehnamGhader, V. Adlakha, M. Mosbach, D. Bahdanau, N. Chapados, and S. Reddy. LLM2Vec: Large language models are secretly powerful text encoders. *arXiv preprint*, 2024.
- [3] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [4] P. Cheng, W. Hao, S. Yuan, S. Si, and L. Carin. Fairfil: Contrastive neural debiasing method for pretrained text encoders. *arXiv preprint arXiv:2103.06413*, 2021.
- [5] T. Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- [6] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022.
- [7] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. Qlora: Efficient fine-tuning of quantized llms. *Advances in Neural Information Processing Systems*, 36, 2024.

- [8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- [9] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.
- [10] D. Y. Fu, T. Dao, K. K. Saab, A. W. Thomas, A. Rudra, and C. Ré. Hungry hungry hippos: Towards language modeling with state space models. *arXiv preprint arXiv:2212.14052*, 2022.
- [11] D. Y. Fu, E. L. Epstein, E. Nguyen, A. W. Thomas, M. Zhang, T. Dao, A. Rudra, and C. Ré. Simple hardware-efficient long convolutions for sequence modeling. In *International Conference on Machine Learning*, pages 10373–10391. PMLR, 2023.
- [12] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, et al. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.
- [13] T. Gao, X. Yao, and D. Chen. Simcse: Simple contrastive learning of sentence embeddings. *arXiv preprint arXiv:2104.08821*, 2021.
- [14] T. Gao, H. Yen, J. Yu, and D. Chen. Enabling large language models to generate text with citations. *arXiv preprint arXiv:2305.14627*, 2023.
- [15] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.
- [16] A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

- [17] A. Gu, T. Dao, S. Ermon, A. Rudra, and C. Ré. Hippo: Recurrent memory with optimal polynomial projections. *Advances in neural information processing systems*, 33:1474–1487, 2020.
- [18] A. Gu, K. Goel, and C. Ré. Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396*, 2021.
- [19] A. Gu, I. Johnson, K. Goel, K. Saab, T. Dao, A. Rudra, and C. Ré. Combining recurrent, convolutional, and continuous-time models with linear state space layers. *Advances in neural information processing systems*, 34:572–585, 2021.
- [20] E. Hernandez, A. S. Sharma, T. Haklay, K. Meng, M. Wattenberg, J. Andreas, Y. Belinkov, and D. Bau. Linearity of relation decoding in transformer language models. *arXiv preprint arXiv:2308.09124*, 2023.
- [21] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [22] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih. Dense passage retrieval for open-domain question answering. *arXiv preprint arXiv:2004.04906*, 2020.
- [23] J. Lee, Z. Dai, X. Ren, B. Chen, D. Cer, J. R. Cole, K. Hui, M. Boratko, R. Kapadia, W. Ding, et al. Gecko: Versatile text embeddings distilled from large language models. *arXiv preprint arXiv:2403.20327*, 2024.
- [24] P. P. Liang, I. M. Li, E. Zheng, Y. C. Lim, R. Salakhutdinov, and L.-P. Morency. Towards debiasing sentence representations. *arXiv preprint arXiv:2007.08100*, 2020.
- [25] S. Massaroli, M. Poli, D. Fu, H. Kumbong, R. Parnichkun, D. Romero, A. Timalsina, Q. McIntyre, B. Chen, A. Rudra, et al. Laughing hyena distillery: Extracting compact

- recurrences from convolutions. *Advances in Neural Information Processing Systems*, 36, 2024.
- [26] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers. Mteb: Massive text embedding benchmark. *arXiv preprint arXiv:2210.07316*, 2022.
- [27] A. Neelakantan, T. Xu, R. Puri, A. Radford, J. M. Han, J. Tworek, Q. Yuan, N. Tezak, J. W. Kim, C. Hallacy, et al. Text and code embeddings by contrastive pre-training. *arXiv preprint arXiv:2201.10005*, 2022.
- [28] J. Ni, C. Qu, J. Lu, Z. Dai, G. H. Ábrego, J. Ma, V. Y. Zhao, Y. Luan, K. B. Hall, M.-W. Chang, et al. Large dual encoders are generalizable retrievers. *arXiv preprint arXiv:2112.07899*, 2021.
- [29] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- [30] L. Peng, Y. Zhang, Z. Wang, J. Srinivasa, G. Liu, Z. Wang, and J. Shang. Answer is all you need: Instruction-following text embedding via answering the question. *arXiv preprint arXiv:2402.09642*, 2024.
- [31] M. Poli, S. Massaroli, E. Nguyen, D. Y. Fu, T. Dao, S. Baccus, Y. Bengio, S. Ermon, and C. Ré. Hyena hierarchy: Towards larger convolutional language models. In *International Conference on Machine Learning*, pages 28043–28078. PMLR, 2023.
- [32] O. Press, N. A. Smith, and M. Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. *arXiv preprint arXiv:2108.12409*, 2021.
- [33] A. S. Sharma, D. Atkinson, and D. Bau. Locating and editing factual associations in mamba. *arXiv preprint arXiv:2404.03646*, 2024.

- [34] H. Su, W. Shi, J. Kasai, Y. Wang, Y. Hu, M. Ostendorf, W.-t. Yih, N. A. Smith, L. Zettlemoyer, and T. Yu. One embedder, any task: Instruction-finetuned text embeddings. *arXiv preprint arXiv:2212.09741*, 2022.
- [35] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- [36] Y. Tay, M. Dehghani, S. Abnar, Y. Shen, D. Bahri, P. Pham, J. Rao, L. Yang, S. Ruder, and D. Metzler. Long range arena: A benchmark for efficient transformers. *arXiv preprint arXiv:2011.04006*, 2020.
- [37] L. Van der Maaten and G. Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9(11), 2008.
- [38] K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt. Interpretability in the wild: a circuit for indirect object identification in gpt-2 small. *arXiv preprint arXiv:2211.00593*, 2022.
- [39] T. Wang and P. Isola. Understanding contrastive representation learning through alignment and uniformity on the hypersphere. In *International conference on machine learning*, pages 9929–9939. PMLR, 2020.
- [40] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [41] Z. Zeng, J. Yu, T. Gao, Y. Meng, T. Goyal, and D. Chen. Evaluating large language models at evaluating instruction following. *arXiv preprint arXiv:2310.07641*, 2023.
- [42] F. Zhang and N. Nanda. Towards best practices of activation patching in language models: Metrics and methods. *arXiv preprint arXiv:2309.16042*, 2023.